Information Retrieval

Ukrainian Catholic University
Lviv, 2025

Sure, here is the recipe of apple pie for you...

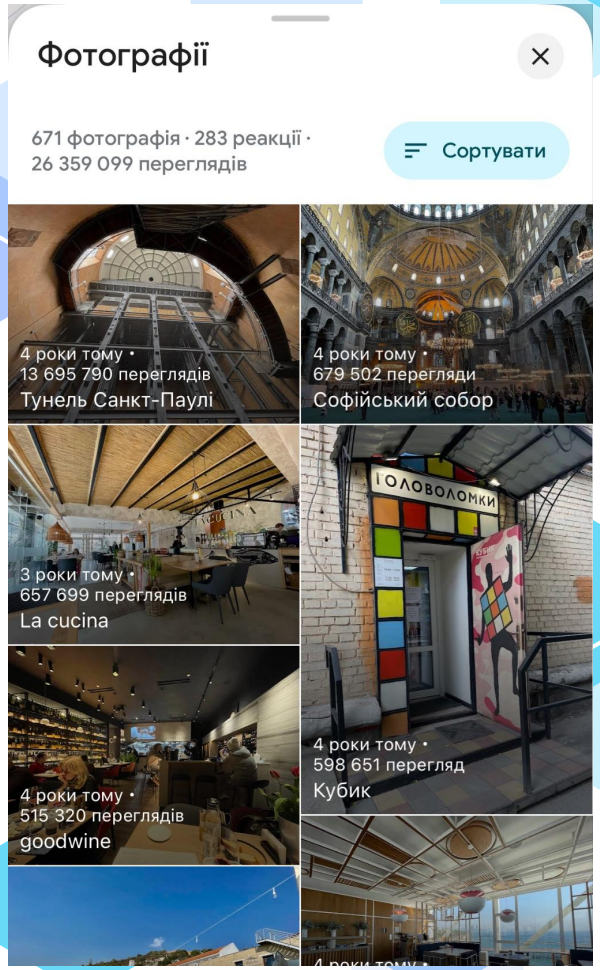
Dmytro Chaplynskyi

Developer, 20+ years of commercial experience; 15 years with Python.

10 years in the field of Ukrainian language processing (NLP); co-organizer of the LANG-UK public initiative. Author/co-author of 10+ articles over the last 3 years, including on the UberText 2.0 and NER UK 2.0 corpora.

Working on creating datasets, LLMs and machine translation.

Second-year graduate student at the Ukrainian Catholic University.



Fun fact about me

I've been to 35 countries so far, saw 3 oceans and crossed the equator once.

I've been to Uganda four times and enjoyed it a lot.

I still maintain Google Maps profile with more than 26 million of views of my photos

So, Information Retrieval (IR)

Just a fancy word for **search**, right?

What is inside the box?

- Document
  Corpus
  Corpora
  Queries
- Inverted index for the efficient retrieval
- Normalization, tokenization, chunking, filtering, and stop words, synonyms
- Stemming or lemmatization
  Dense representations!
- Ranking: TF-IDF, BM25, Cosine similarity and proxies
  But also pagerank, re-ranking, etc
- Metrics: P@k, MAP/MRR, nDCG@k

Let’s unpack: key entities

Document

The **atomic unit** of content in IR/NLP that you index and retrieve—e.g., a web page, article, paragraph, tweet, or any defined text chunk.

Corpus

A curated **collection** of documents prepared for analysis or search, often with **consistent formatting**, metadata, and possibly annotations.

Corpora

Plural of corpus.

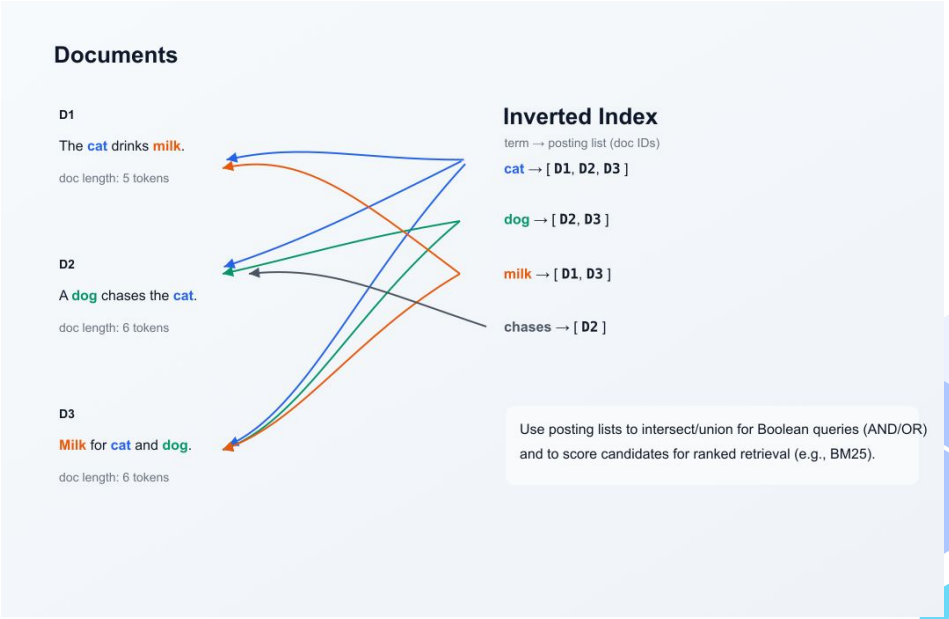
Query

user’s expression of an information need—the text (keywords or a natural-language question) submitted to retrieve relevant documents. Often comes with **Domain Specific Language** (DSL)

Let’s unpack: Inverted index

An **inverted index** is the search engine’s **core structure** that **flips** the usual view of documents: instead of **storing documents → terms**, it stores **each term → list of document IDs** (and often term frequencies and positions).

At query time, the engine looks up the posting lists for the query terms and **combines them** (intersections/unions) to find candidates **quickly**, then optionally **scores** them (e.g., BM25) or checks positions for phrases—avoiding a full scan of all documents.



Let’s unpack: Preprocessing

Normalization

Remove markup or non-printed characters, normalize apostrophes, dashes and diacritics. Expand contractions (don’t -> do not) Same applies to query!

Tokenization

Break text into tokens (basically words, but also URLs, numbers). Not as easy as it seems!

Chunking

Break the text into smaller chunks for the RAG, but do it in a smart way!

Filtering & stopwords

Remove articles like A and The, conjunctions, and other high-frequency words that convey little or no sense.

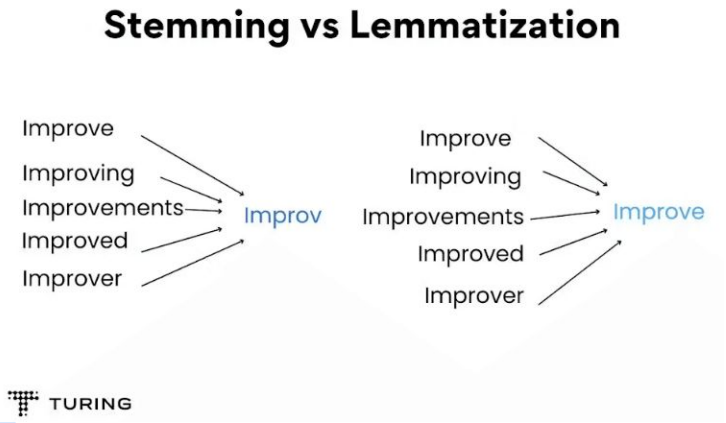
Let’s unpack: Stemming/lemmatization

Stemming

Algorithmical removal of inflected parts that returns the “stem” of the word: argue, argued, argues, arguing, and argus-> argu

Lemmatization

Harder to implement, reduces the word to its basic form, “lemma” Usually operates on a wordform dictionary

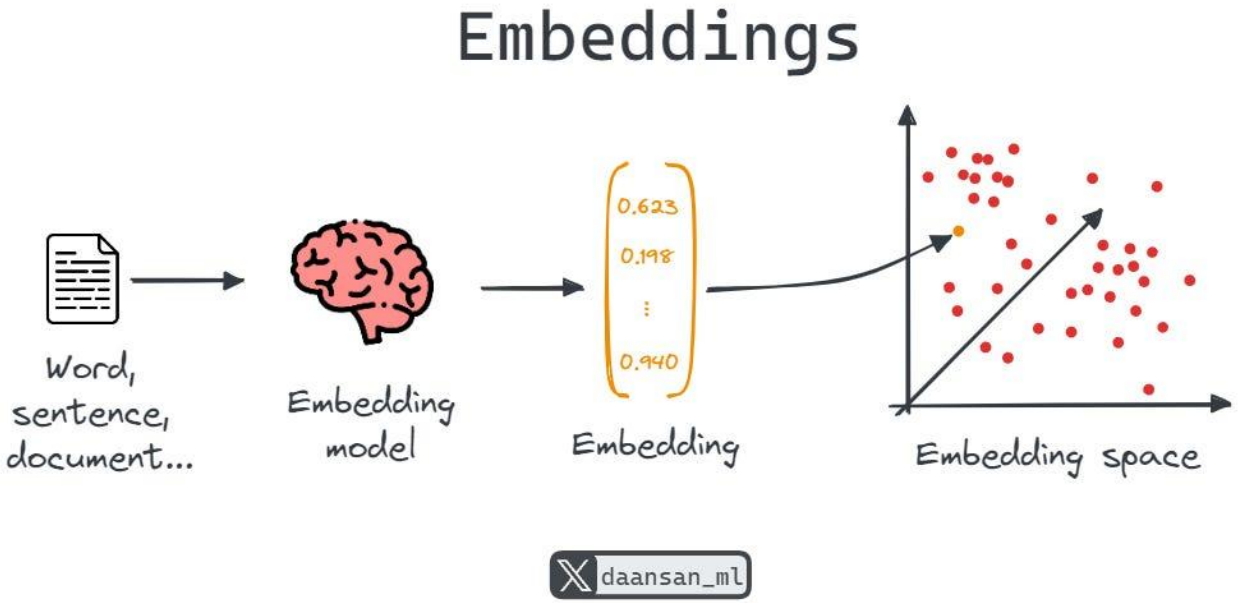


привіт – іменник чоловічого роду

відмінок	однина	множина
називний	привіт	привіти
родовий	привіту	привітів
давальний	привіту, привітові	привітам
знахідний	привіт	привіти
орудний	привітом	привітами
місцевий	на/у привіті	на/у привітах
кличний	привіте*	привіти*

Let’s unpack: Dense representation

Convert a text or other modality into a vector



Let’s unpack: Dense representation pt2

```python
from sentence_transformers import SentenceTransformer

# 1. Load a pretrained Sentence Transformer model
model = SentenceTransformer("all-MiniLM-L6-v2")

# The sentences to encode
sentences = [
    "The weather is lovely today.",
    "It's so sunny outside!",
    "He drove to the stadium.",
]

# 2. Calculate embeddings by calling model.encode()
embeddings = model.encode(sentences)
print(embeddings.shape)
# [3, 384]

# 3. Calculate the embedding similarities
similarities = model.similarity(embeddings, embeddings)
print(similarities)
# tensor([[1.0000, 0.6660, 0.1046],
#         [0.6660, 1.0000, 0.1411],
#         [0.1046, 0.1411, 1.0000]])
```

Let's unpack: TF-IDF

TF-IDF (term frequency–inverse document frequency): Weighs a term by how often it appears in a document (**tf**) and down-weights it by how common it is across the corpus (**idf**). Ranking is typically done with **cosine similarity** between TF-IDF vectors, making it a fast, strong baseline that handles vocabulary mismatch poorly and doesn't saturate with very high tf.

$$w_{x,y} = tf_{x,y} \times \log\left(\frac{N}{df_x}\right)$$

TF-IDF

Term x within document y

$tf_{x,y}$ = frequency of x in y

df_x = number of documents containing x

N = total number of documents

Let's unpack: BM25

BM25 (Okapi): A probabilistic ranking function that sums per-term contributions with two key tweaks: **term-frequency saturation** (extra repeats help less and less) and document-length normalization. In practice, BM25 is the default workhorse for lexical search because it balances tf and length robustly.

sum the scores for each query term

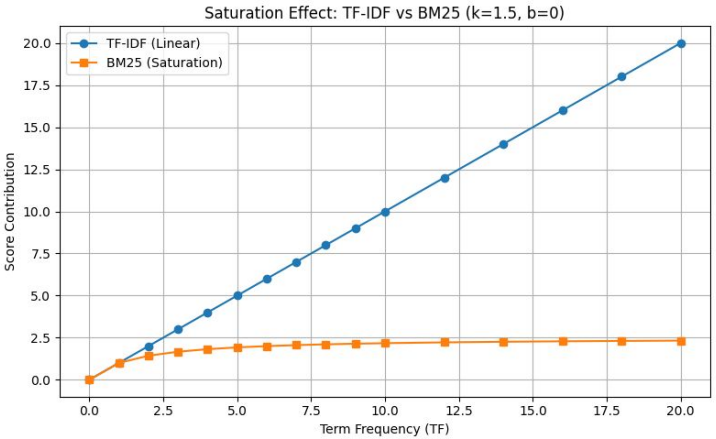
forget about this: it doesn't affect score relationships so Lucene took it out

probabilistic flavor of IDF: Lucene adds a 1 inside the log, making it basically the same as traditional IDF

$$\text{score}(D,Q) = \sum_{t \in Q} \frac{f_{t,D} \cdot (k_1 + 1)}{f_{t,D} + k_1 \cdot (1 - b + b \cdot \frac{|D|}{\text{avgdl}})} \cdot \log \frac{N - n_t + 0.5}{n_t + 0.5}$$

term frequency saturation trick

adjust saturation curve based on document length



Let’s unpack: Cosine Similarity & proxies

Cosine similarity:

$$\text{cosine similarity} = S_C(A, B) := \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2} \cdot \sqrt{\sum_{i=1}^{n} B_i^2}},$$

Now assume that you got **10 million of documents** in your index!

Solution? Narrow the scope

For **sparse vectors** you can just ignore all the documents which has nothing in common with the query, like not a single token. That'll remove a lot of computations

But sometime it is not enough. Especially if your work with **dense vectors**. For this one you have to use **Approximate Nearest Neighbor (ANN)**. These methods prioritize speed over accuracy, but allow you to extract relevant candidates much faster.

- **Local Sensitivity Hashing**, or **LSH**. Computes compact hashes using hash functions (such as **minhash** for texts or random projections for dense vectors) and allow to quickly compare hashes.
- **Hierarchical Navigable Small World** or HNSM. Builds a graph, similar to binary search, which allows you to quickly find a leaf full of relevant documents by doing just a tiny number of comparisons.

Finally, metrics

Precision = TP / (TP+FP): “of what I retrieved, how much was relevant?”
Precision([R, N, N, R, R, N, N, N, R, N]) = ?

Recall = TP / (TP+FN): “of all relevant items, how many did I retrieve?”
Assume we have 5 relevant items, in ground truth. Recall([N, N, N, R, N, N, N, N]) = ?

P@k (Precision at k) = (# relevant in top k) / k: The fraction of the top-k results that are relevant
P@5([R, N, N, R, R, N, N, N, R, N]) = ?

AP (Average Precision): For one query, average the precision values computed at each rank where a relevant item occurs
AP([R, N, N, R, R, N, N, N, R, N]) = ?

...still metrics

MAP (Mean Average Precision) = avg(AP({Q1, Q2, ... QN}))
Q_1 top-5 = [R, N, R, N, R]
Q_2 top-3 = [N, R, N]
AP1 = ? AP2 = ? MAP = ?

MRR (Mean Reciprocal Rank) = avg(1 / (rank of the first relevant result))
Basically, how often the relevant document appears at the top of results

Assume that for 3 queries we have first relevant at ranks [1, 3, NONE].
RR = ?
MRR = ?

...guess what? Still metrics!

nDCG@k (normalized Discounted Cumulative Gain at k)

Computes DCG@k, then normalize by the ideal DCG

$$\mathrm{DCG_p} = \sum_{i=1}^{p} \frac{rel_i}{\log_2(i+1)} = rel_1 + \sum_{i=2}^{p} \frac{rel_i}{\log_2(i+1)}$$

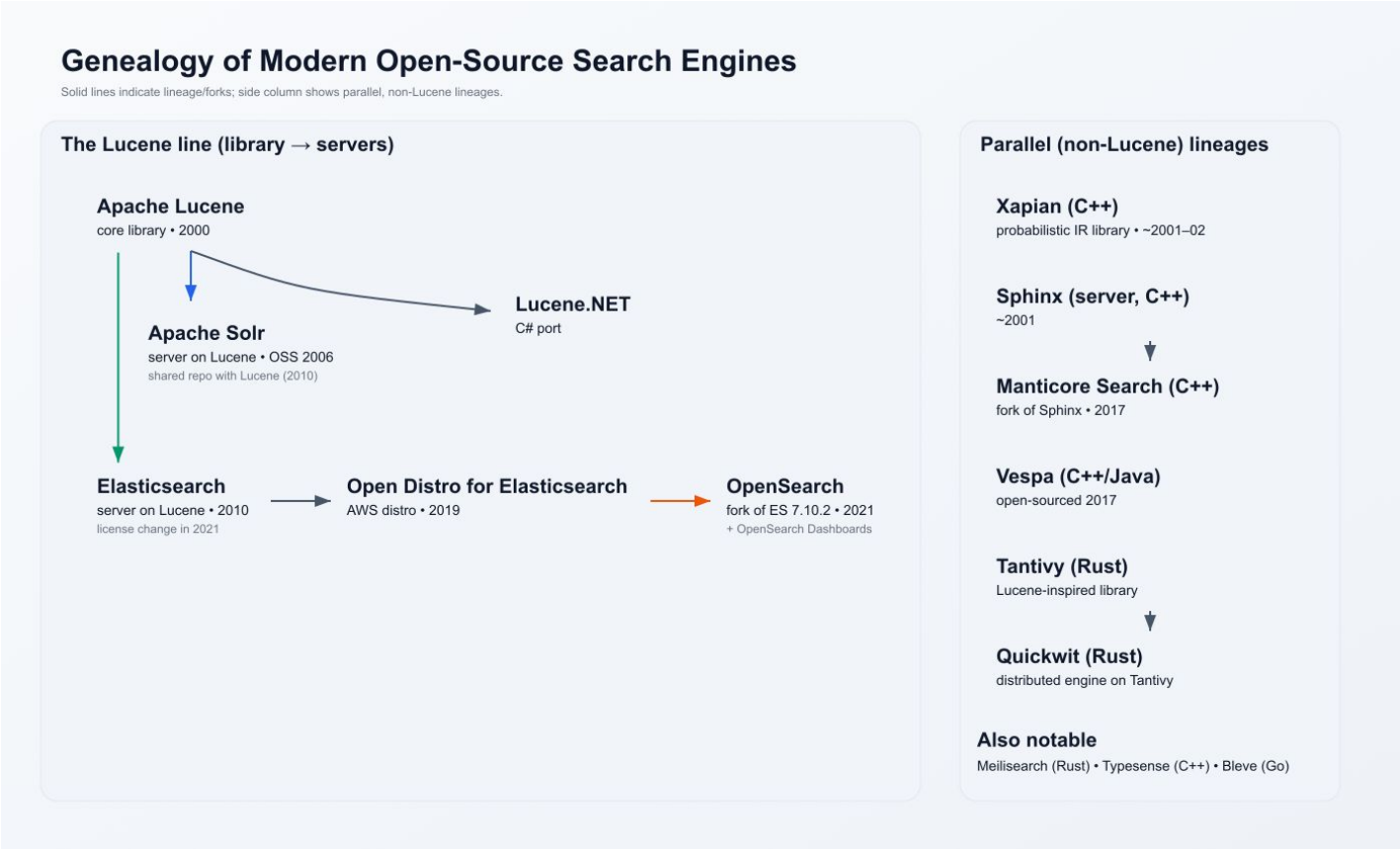
Toy example (k = 5).

Suppose the top-5 results have graded labels $[2, 3, 0, 1, 2]$.

rank i	rel_i	discount $\log_2(i+1)$	contribution = rel/discount
1	2	1.000	2.000
2	3	1.585	1.893
3	0	2.000	0.000
4	1	2.322	0.431
5	2	2.585	0.774

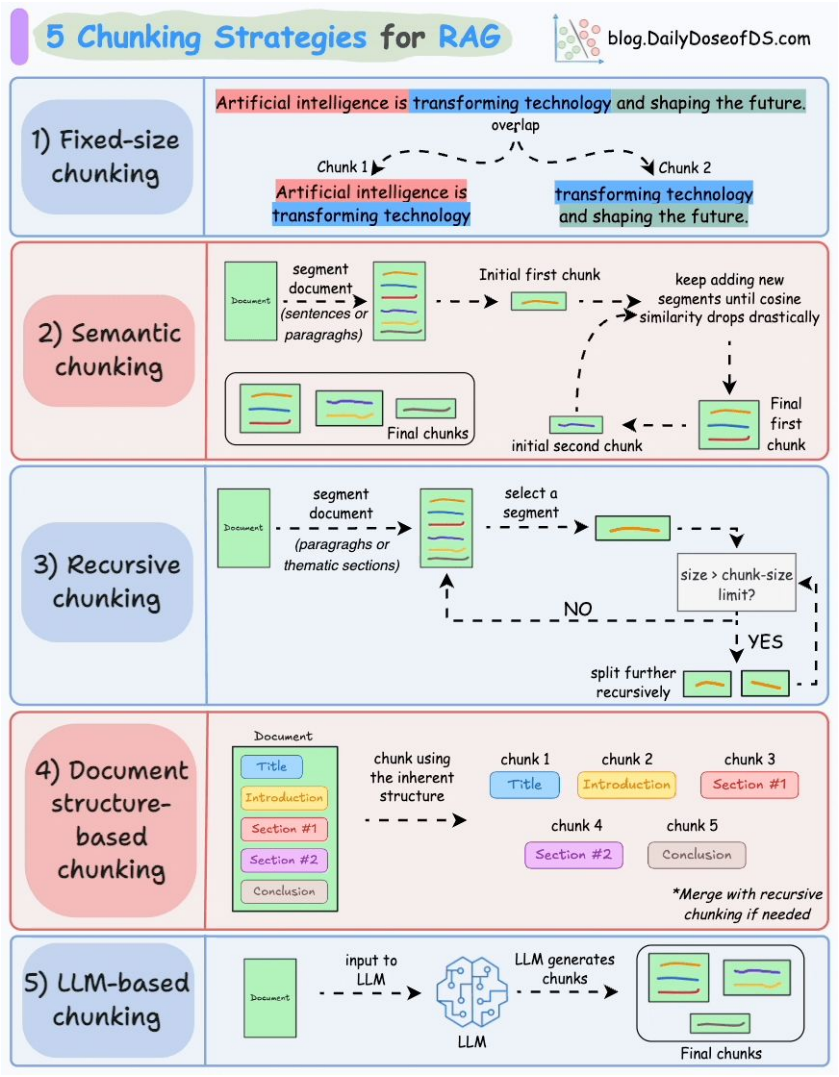
- **DCG@5** $\approx 2.000 + 1.893 + 0 + 0.431 + 0.774$ **= 5.097**.
- The same labels in **ideal order** are $[3, 2, 2, 1, 0]$:
  - **IDCG@5** $= 3/1 + 2/1.585 + 2/2 + 1/2.322 + 0$ **= 5.693**.
- **nDCG@5** $= 5.097/5.693 \approx 0.895$.

Genealogy of OSS Search Engines



Bonus slide: Chunking For RAG

Slice long document wisely, so sentence embeddings works well and you can retrieve relevant fragments using Cosine Similarity and friends



Questions, maybe?

Thanks!

Contact me

chaplynskyi.dmytro@ucu.edu.ua
@dchaplinsky on twitter/tg

https://github.com/lang-uk/
https://huggingface.co/lang-uk